

ADAPTIVE DEEP NEURAL NETWORK INFERENCE USING CONFIDENCE-BASED EARLY EXITS

**Interim Report on
Deep Learning Project
[ICT-4442]**

Submitted By

**Aditya Goyal | 230953554
Raunak Panshikar | 230953326
Arth Mishra | 230911568
Kartikey Goyal | 230911536**



MANIPAL
ACADEMY *of* HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

**SCHOOL OF COMPUTER ENGINEERING
MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL ACADEMY OF HIGHER EDUCATION**

OCTOBER 2026

PART B: INTERIM REPORT

Team No. and Names (with Registration Numbers):

Team No.: 15

Aditya Goyal (230953554)

Raunak Panshikar (230953326)

Arth Mishra (230911568)

Kartikey Goyal (230911536)

Title of the Project:

Adaptive Deep Neural Network Inference Using Confidence-Based Early Exits

Code repository:

<https://github.com/goyaladitya05/adaptive-dnn-inference>

1. Project Overview and Interim Status

Standard deep networks spend the same computation on every input, although many images can be classified correctly by a fraction of the network. This project builds early-exit networks for CIFAR-100: classifiers are attached at several depths of a backbone, and at inference time an image leaves the network at the first exit whose prediction satisfies a confidence criterion. We compare four models: (1) a standard ResNet-18 with full inference, (2) a confidence-based early-exit ResNet-18 using the maximum softmax probability, (3) a calibrated and entropy-based variant that applies per-exit temperature scaling before thresholding, and (4) an early-exit ViT-Tiny. For each model we measure accuracy, average computation (FLOPs), real batch-1 latency on a GPU and on a CPU, calibration, and the behaviour of the exit policies under noise, blur and contrast corruptions.

At this interim stage, the data pipeline (including corrupted test sets) and all four models are implemented and trained once with preliminary, untuned settings on Kaggle T4 GPUs. Because Models 2 and 3 deliberately share one trained network and differ only in the exit rule, three training runs cover all four models. Table 1 summarises the status of each deliverable.

Table 1: *Interim status of project deliverables.*

Deliverable	Status	Where
Literature review (10 papers)	Complete	Section 2, Table 2
Dataset acquisition and preprocessing	Complete	Section 3; src/data.py, src/corruptions.py
Model 1: ResNet-18 baseline	Implemented, trained	Sections 4.2, 5
Model 2: Confidence EE-ResNet-18	Implemented, trained	Sections 4.3, 5
Model 3: Calibrated / entropy EE-ResNet-18	Implemented, evaluated	Sections 4.4, 5
Model 4: EE-ViT-Tiny	Implemented, trained (preliminary)	Sections 4.5, 5
Latency benchmark (GPU and CPU, batch 1)	Implemented, first measurements	Section 5.5
Corruption robustness study	Pipeline complete, preliminary results	Section 5.6
Risks and plan to final submission	Complete	Section 6

2. Literature Review

We reviewed ten papers covering early-exit CNNs [1], [2], [4], [8], input-dependent routing [3], [6], calibration [5], [10], corruption robustness [7] and early exiting in vision transformers [9]. Table 2 summarises each paper; all quantitative results were checked against the original papers.

Table 2: *Summary of reviewed literature.*

#	Paper	Method	Dataset(s)	Key result	Relevance to this project
[1]	Teerapittayanon et al., ICPR 2016 (BranchyNet)	Side-branch classifiers on LeNet, AlexNet and ResNet-110, trained jointly; a sample exits at the first branch whose softmax entropy is below a tuned	MNIST, CIFAR-10	B-ResNet-110 (CIFAR-10): 1.9x measured speed-up on CPU and GPU, accuracy 79.2% vs 80.7%; B-LeNet: 5.4x CPU speed-up, 94% of samples exit early.	Origin of the entropy exit criterion used in Model 3; one of few works reporting measured latency rather than FLOPs.

#	Paper	Method	Dataset(s)	Key result	Relevance to this project
		threshold.			
[2]	Huang et al., ICLR 2018 (MSDNet)	Multi-scale dense network keeping coarse and fine features throughout, with intermediate classifiers; exits when the maximum softmax probability exceeds a threshold.	CIFAR-10, CIFAR-100, ImageNet	CIFAR-100 budgeted batch: matches ResNet-110 accuracy with about 1/10 of its FLOPs; ImageNet anytime: about 4-8% higher accuracy than ResNet/DenseNet ensembles at low budgets.	Standard CIFAR-100 multi-exit baseline; shows intermediate classifiers can reduce a ResNet's final accuracy (up to 7%), a risk for our ResNet-18.
[3]	Wang et al., ECCV 2018 (SkipNet)	Lightweight gates skip residual blocks per input; trained with supervised pre-training followed by hybrid reinforcement learning.	CIFAR-10, CIFAR-100, SVHN, ImageNet	Computation reduced by 30-90% at preserved accuracy; for the deepest ResNets, savings of 37% (CIFAR-100), 50% (CIFAR-10), 86% (SVHN), 30% (ImageNet).	Layer-skipping alternative to early exiting; positions our exit-based design among dynamic-depth methods for ResNets.
[4]	Kaya et al., ICML 2019 (SDN)	Internal classifiers added to VGG-16, ResNet-56, WRN-32-4 and MobileNet; exits on maximum softmax confidence; characterises network "overthinking".	CIFAR-10, CIFAR-100, Tiny ImageNet	Confidence exits cut average inference cost (FLOPs) by over 50% on CIFAR-10/100 while preserving accuracy; VGG-16-SDN on CIFAR-100: 72.5% at $\leq 50\%$ cost vs 70.9%.	Closest precedent for Model 2: the same max-softmax exit rule on CIFAR-100 ResNets.
[5]	Guo et al., ICML 2017	Study of calibration in modern networks; compares histogram binning, isotonic regression, BBQ, and matrix, vector and temperature scaling as post-hoc fixes.	CIFAR-10/100, ImageNet, SVHN, CUB, Cars, 20 News, Reuters, SST	Modern networks are poorly calibrated; temperature scaling reduces ResNet-110 CIFAR-100 ECE from 16.53% to 1.26% without changing accuracy.	Provides ECE and temperature scaling, applied per exit in Model 3 so that one threshold means the same at every exit.
[6]	Wu et al., CVPR 2018 (BlockDrop)	A policy network trained with reinforcement learning decides in one shot which residual blocks of a pretrained ResNet to execute per image.	CIFAR-10, CIFAR-100, ImageNet	ResNet-101 on ImageNet: 20% average speed-up (in FLOPs) at unchanged 76.4% top-1; ResNet-110 on CIFAR-100: 73.7% using 55% of blocks.	Its sequential-policy variant ran slower than the full ResNet despite fewer blocks: FLOP savings need not imply wall-clock savings.
[7]	Hendrycks and Dietterich, ICLR 2019	Robustness benchmark of 15 algorithmic corruptions (noise, blur, weather, digital) at five severities, summarised by mean Corruption Error (mCE).	ImageNet-C/P, CIFAR-10-C, Tiny ImageNet-C (CIFAR-100-C in the same repository)	Relative corruption robustness barely changed from AlexNet to ResNet; ResNet-50 mCE 76.7%, MSDNet 73.6%.	Defines the corruption types and severity constants we re-implement to stress-test exit policies.
[8]	Wolczyk et al., NeurIPS 2021 (ZTW)	Cascade connections pass each internal classifier's output to the next; a weighted geometric ensemble of past predictions feeds the confidence-based exit decision.	CIFAR-10, CIFAR-100, Tiny ImageNet, ImageNet	ResNet-56 on CIFAR-100 at 50% of base FLOPs: 62.1% accuracy vs 57.2% (SDN) and 53.5% (PABEE).	Strong confidence-exit baseline on CIFAR-100; shows discarded early predictions carry information an exit policy can reuse.
[9]	Xu et al., ACM MM 2023 (LGViT)	Early-exit ViTs with heterogeneous heads (local perception, global aggregation), two-stage training with self-distillation, max-softmax exits.	CIFAR-100, Food-101, ImageNet-1K	About 1.8x average speed-up for about 2% accuracy loss; ViT-B/16 on CIFAR-100: 88.5% vs 90.8% at 1.87x (computed from exit layers).	Closest vision-transformer precedent for Model 4; its speed-up is derived from exit depth, not measured latency.
[10]	Meronen et al., WACV 2024	Post-hoc last-layer Laplace approximation at each exit plus model-internal ensembling and tuned temperature, for uncertainty-aware exits in MSDNet.	CIFAR-100, ImageNet, Caltech-256	CIFAR-100 (small MSDNet, averaged over budgets): ECE falls from 0.182 to 0.017 and top-1 rises from 69.25% to 69.84%.	Direct evidence that early-exit confidences are overconfident and that calibrating them helps exit decisions; motivates Model 3.

2.1 Discussion

Early-exit networks. BranchyNet [1] introduced jointly trained side branches and an entropy threshold, reporting a measured 1.9x speed-up for a branchy ResNet-110 on CIFAR-10 at a cost of 1.5 points of accuracy. MSDNet [2] showed that naive intermediate classifiers can lower a ResNet's final accuracy by up to 7% on CIFAR-100, and addressed this with a multi-scale dense architecture that matches ResNet-110 with about one tenth of the computation. Shallow-Deep Networks [4] attached internal classifiers to off-the-shelf networks, identified overthinking (correct intermediate predictions that later become wrong), and cut average FLOPs by more than half with a max-softmax threshold. Zero Time Waste [8] reuses the predictions of exits that did not fire through cascade connections and ensembling, improving the accuracy of confidence-based exits on CIFAR-100 at a fixed budget.

Dynamic routing. SkipNet [3] and BlockDrop [6] make depth input-dependent by skipping residual blocks using learned gates or a policy network. They report 30-90% and 20% FLOP reductions

respectively, but BlockDrop’s own timing shows that a sequential decision variant can run slower than the unmodified network, an early warning that FLOP counts and wall-clock time can disagree.

Calibration and robustness. Guo et al. [5] showed that modern networks are overconfident and that a single temperature fitted on validation data almost removes the calibration error without changing predictions. Meronen et al. [10] found the same overconfidence in early-exit networks and showed that calibrating each exit improves both ECE and accuracy at a given budget. Hendrycks and Dietterich [7] defined the corruption benchmark (15 corruption types, 5 severities) that we follow. LGViT [9] brought early exits to vision transformers, showing that simple heads on shallow transformer blocks are weak and require specialised heads and self-distillation.

2.2 Research Gap

Most early-exit work decides with raw softmax confidence or entropy [1], [2], [4], [8], [9] without an explicit calibration step, although networks, and early exits in particular, are overconfident [5], [10]. Efficiency is mostly reported as FLOPs or exit depth [2]-[4], [8]-[10], although decision overhead can erase theoretical savings [6]. Finally, none of the reviewed early-exit methods studies how exit thresholds, exit distributions and calibration behave when inputs are corrupted [7]. This project addresses all three points on a common CIFAR-100 setup: it compares raw and calibrated exit rules, reports measured latency next to FLOPs, and evaluates every exit policy under noise, blur and contrast corruptions.

3. Dataset Acquisition and Preprocessing Pipeline

3.1 Dataset

CIFAR-100 [11] contains 60,000 RGB images of 32x32 pixels in 100 classes (600 per class), split into 50,000 training and 10,000 test images. It is downloaded programmatically through torchvision, so every run (local or on Kaggle) obtains identical data. We hold out a stratified validation set of 5,000 training images (50 per class, fixed seed 0) that is used for model selection, fitting temperatures and choosing exit thresholds. The official test set is used only for reporting. Figure 1 shows random test samples.

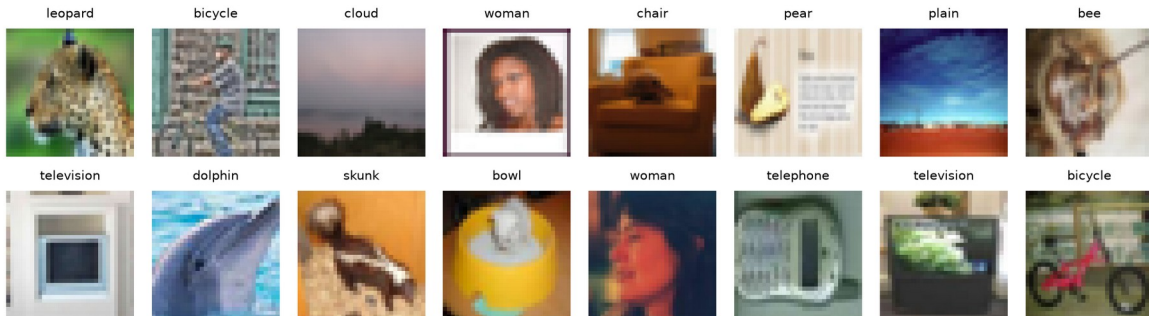


Figure 1: Random CIFAR-100 test images with their class labels.

Table 3: Data splits.

Split	Images	Per class	Used for
Train	45,000	450	Gradient updates
Validation	5,000	50	Monitoring, temperature fitting, threshold selection
Test (clean)	10,000	100	Final accuracy, compute, calibration, latency
Test (corrupted)	6 x 5 x 10,000	100	Robustness of exit policies (Section 3.3)

3.2 Preprocessing and Augmentation

Kaggle machines provide two T4 GPUs but only four CPU cores, so a standard CPU data loader with per-image PIL transforms becomes the bottleneck for small 32x32 images. Instead, the full uint8 dataset (about 150 MB) is copied to GPU memory once, and batches are sampled, augmented and normalised on the GPU. Training batches use the standard light CIFAR augmentation: random 32x32 crops from a 4-pixel reflect-padded image and random horizontal flips. Images are scaled to [0, 1] and normalised with the per-channel CIFAR-100 training mean (0.507, 0.487, 0.441) and standard deviation (0.267, 0.256, 0.276), which we verified on our training split. The ViT-Tiny additionally uses mixup [15] or CutMix [16]

on every batch and label smoothing of 0.1, since transformers trained from scratch on small datasets need stronger regularisation. Evaluation uses no augmentation.

```
def load_cifar100(root="data", val_size=5000, seed=0):
    tr = CIFAR100(root, train=True, download=True)
    te = CIFAR100(root, train=False, download=True)
    x, y = tr.data, np.asarray(tr.targets)
    rng = np.random.default_rng(seed)
    per_class = val_size // NUM_CLASSES # 50 images per class
    val_idx = np.concatenate([rng.permutation(np.flatnonzero(y == c))[:per_class]
                              for c in range(NUM_CLASSES)])

    mask = np.ones(len(y), dtype=bool)
    mask[val_idx] = False
    return {"train": (x[mask], y[mask]),
            "val": (x[val_idx], y[val_idx]),
            "test": (te.data, np.asarray(te.targets))}

def augment(x, pad=4):
    """Random crop with reflect padding and horizontal flip, on the GPU."""
    n, _, h, w = x.shape
    x = F.pad(x, (pad, pad, pad, pad), mode="reflect")
    i = torch.randint(0, 2 * pad + 1, (n, 1), device=x.device) + arange(h)
    j = torch.randint(0, 2 * pad + 1, (n, 1), device=x.device) + arange(w)
    x = x[arange(n)[: , None, None, None], arange(3)[None, :, None, None],
          i[: , None, :, None], j[: , None, None, :]]
    flip = torch.rand(n, device=x.device) < 0.5
    return torch.where(flip[: , None, None, None], x.flip(3), x)

class GPULoader:
    # the whole uint8 split lives on the GPU
    def __iter__(self):
        idx = torch.randperm(n) if self.train else torch.arange(n)
        for k in range(0, n, self.bs):
            b = idx[k:k + self.bs]
            xb = self.x[b].float().div_(255)
            if self.train:
                xb = augment(xb)
            if self.transform is not None: # corruptions, Section 3.3
                xb = self.transform(xb)
            yield normalize(xb), self.y[b]
```

3.3 Corrupted Test Sets

To study whether exit decisions stay reliable under distribution shift, we re-implemented six corruptions of the CIFAR-100-C benchmark [7] in PyTorch so they run on the GPU: Gaussian, shot (Poisson) and impulse (salt-and-pepper) noise, Gaussian and defocus blur, and contrast reduction, each at five severities with the constants of the original CIFAR-C generation code. Corrupted images are re-quantised to 8 bits, as in the stored benchmark, and every corruption and severity uses a fixed random seed, so all models see identical corrupted images. This gives 30 corrupted copies of the test set (300,000 images) without storing any extra data. Figure 2 shows one test image under each corruption.

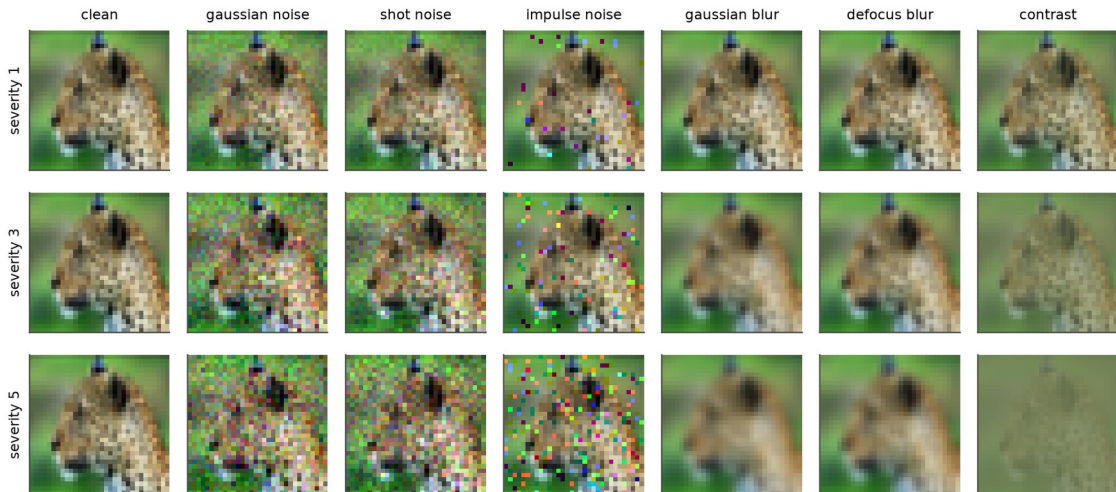


Figure 2: One test image under the six corruptions at severities 1, 3 and 5.

```

def gaussian_noise(x, s, g):
    c = [0.04, 0.06, 0.08, 0.09, 0.10][s - 1]
    return (x + torch.randn(x.shape, generator=g, device=x.device) * c).clamp(0, 1)

def gaussian_blur(x, s, g):
    c = [0.4, 0.6, 0.7, 0.8, 1.0][s - 1]
    k = _gauss1d(c, int(4 * c + 0.5), x.device)
    return _filter(x, torch.outer(k, k), "replicate").clamp(0, 1)

def corrupt(name, severity, generator):  # re-quantize to 8 bits like stored CIFAR-C
    fn = CORRUPTIONS[name]
    return lambda x: torch.round(fn(x, severity, generator) * 255) / 255

```

4. Models and Training Setup

4.1 Common Multi-Exit Design

All models share one abstraction: a backbone split into K segments with a classifier head after each segment. A plain network is the special case $K = 1$. Training minimises the equally weighted mean of the cross-entropy losses of all exits, so a single run trains every exit. At inference, the image passes through segment k , head k produces logits, and the image exits if its score clears a threshold; otherwise it continues. Our implementation removes exited images from the batch before the next segment, so savings are real and not only simulated. Stopping at exit k costs the backbone up to segment k plus every head evaluated so far.

```

@torch.no_grad()
def adaptive_forward(model, x, score, thresh, temps=None):
    """Real early-exit inference: exited samples are dropped before the next segment."""
    alive, h = torch.arange(len(x)), x
    for k in range(model.num_exits):
        h = model.segments[k](h)
        logits = model.heads[k](h)
        if k == model.num_exits - 1:
            preds[alive] = logits.argmax(1); break
        done = score(logits, temps[k]) >= thresh  # max_prob or 1 - H(p)/log C
        preds[alive[done]] = logits[done].argmax(1); exit_at[alive[done]] = k
        alive, h = alive[~done], h[~done]

```

4.2 Model 1: ResNet-18 Baseline

We use the CIFAR variant of ResNet-18 [12]: a 3x3 convolution stem without max-pooling followed by four stages of two basic residual blocks (64, 128, 256 and 512 channels; spatial sizes 32, 16, 8 and 4), global average pooling and a linear classifier. It has 11.22 M parameters and needs 1.11 GFLOPs per image. Every image uses the full network.

4.3 Model 2: Confidence-Based Early-Exit ResNet-18

The same backbone receives an exit after each of the four stages. Exit heads for stages 1 to 3 use strided 3x3 convolutions (128 channels, batch normalisation, ReLU) to reduce the feature map to 4x4, followed by global average pooling and a linear layer; the final exit is the standard pooled classifier. Deeper exits need fewer downsampling steps, so heads become cheaper with depth. An image exits at exit k when the maximum softmax probability of head k is at least a threshold τ , the rule of SDN [4] and MSDNet [2].

Table 4: Cost of stopping at each exit of EE-ResNet-18 (12.22 M parameters). Heads add 8.5% to the cost of a full pass.

Exit	Position	Cumulative GFLOPs	Relative to ResNet-18
Exit 1	after stage 1	0.367	33.0%
Exit 2	after stage 2	0.659	59.3%
Exit 3	after stage 3	0.937	84.3%
Exit 4	after stage 4	1.205	108.5%

4.4 Model 3: Calibrated / Entropy-Based Early-Exit ResNet-18

Model 3 reuses the trained network of Model 2 and changes only the exit rule, which isolates the effect of the decision criterion from training noise. First, each exit k is calibrated with temperature scaling [5]: a scalar T_k is fitted on the validation set by minimising negative log-likelihood, and logits are divided by

T_k before the softmax. This does not change any prediction, but it puts the confidences of different exits on a common, calibrated scale, so a single threshold means the same thing at every exit. Second, as an alternative to the maximum probability, we use the normalised entropy score $1 - H(p)/\log(100)$, which uses the whole predictive distribution, following BranchyNet [1]. We evaluate four rules: max-prob and entropy, each with raw and temperature-scaled (TS) logits.

4.5 Model 4: Early-Exit ViT-Tiny

The transformer follows the ViT-Tiny configuration [13], [14] (embedding width 192, 12 blocks, 3 attention heads, MLP ratio 4) adapted to 32x32 inputs with 4x4 patches, giving 64 tokens and learned position embeddings. Instead of a class token, each exit applies LayerNorm, averages the tokens and uses a linear classifier. Exits are placed after blocks 3, 6, 9 and 12, so exit costs grow linearly with depth and heads are almost free. Stochastic depth with rate up to 0.1 is used during training. It has 5.44 M parameters and needs 0.72 GFLOPs for a full pass.

4.6 Training Configuration

Table 5: Preliminary training settings (not yet tuned).

Setting	ResNet-18 and EE-ResNet-18	EE-ViT-Tiny
Optimiser	SGD, momentum 0.9, Nesterov	AdamW (0.9, 0.999)
Learning rate	0.1, 2 warmup epochs, cosine decay	1e-3, 5 warmup epochs, cosine decay
Weight decay	5e-4 (not on BN / bias)	0.05 (not on norm / bias / position)
Batch size, epochs	128, 80	256, 100
Augmentation	random crop, flip	random crop, flip, mixup / CutMix
Regularisation	none	label smoothing 0.1, drop path 0.1, gradient clip 1.0
Loss	mean cross-entropy over exits	mean cross-entropy over exits
Precision, hardware	fp16 mixed precision, Kaggle T4	fp16 mixed precision, Kaggle T4

The baseline and EE-ResNet-18 were trained in parallel on the two T4 GPUs of one Kaggle session and the ViT in a second session. Each Kaggle job clones the repository at a given commit, downloads the data, trains, and then runs the evaluation script, which stores the logits of every exit for the validation set, the clean test set and all 30 corrupted test sets. All exit policies are then analysed offline from these logits, so new thresholds or rules need no further GPU time.

4.7 Evaluation Protocol

- **Accuracy and compute.** Top-1 test accuracy and mean FLOPs per image, counted with PyTorch's FlopCounterMode including all heads that an image passes.
- **Threshold selection.** For each rule, the threshold is chosen on the validation set as the cheapest one whose validation accuracy is within 1 point of the same model's final exit; test numbers are reported at that threshold. Full accuracy and compute curves over all thresholds are also reported.
- **Calibration.** Expected calibration error (ECE, 15 bins) per exit, before and after temperature scaling.
- **Latency.** Mean time per image when images arrive one at a time (batch 1), using real early-exit inference, on a Kaggle T4 GPU and on one CPU thread, after warm-up.
- **Robustness.** Accuracy, mean compute and accuracy of early-exited images on the 30 corrupted test sets, using thresholds selected on clean validation data.

5. Preliminary Results

All numbers below come from a single training run per model with untuned hyperparameters and should be read as preliminary. Figure 3 shows the validation accuracy of every exit during training.

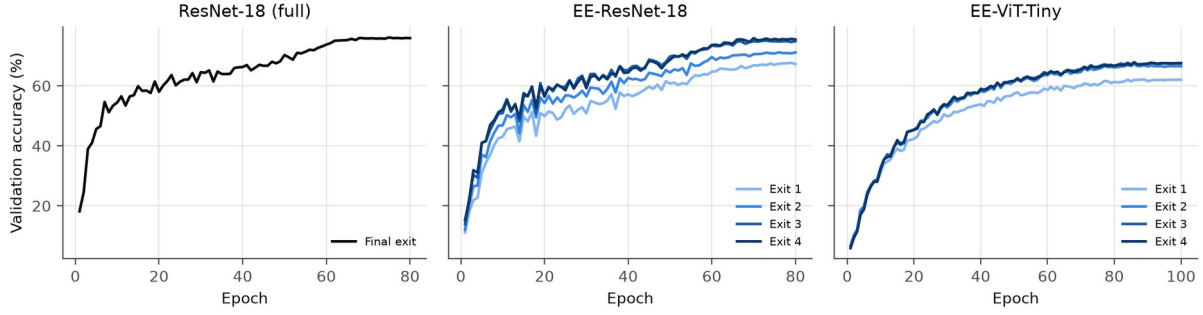


Figure 3: Validation accuracy during training (one curve per exit).

5.1 Accuracy of Each Model and Exit

Table 6: Top-1 test accuracy (%) of every exit when all images are forced through that exit.

Model	Params (M)	GFLOPs (full)	Exit 1	Exit 2	Exit 3	Final exit
ResNet-18 (Model 1)	11.22	1.11	-	-	-	75.22
EE-ResNet-18 (Models 2, 3)	12.22	1.21	69.11	71.62	75.35	75.88
EE-ViT-Tiny (Model 4)	5.44	0.72	62.52	66.33	66.62	66.53

The baseline ResNet-18 reaches 75.22% test accuracy. The final exit of EE-ResNet-18 reaches 75.88%, +0.66 points relative to the baseline, so jointly training the early heads did not harm the full network in our setting, possibly because the extra supervision at intermediate depths acts as a regulariser; this contrasts with the degradation of up to 7% reported for naive intermediate classifiers in [2]. The early exits are strong: exit 1 reaches 69.1% with 33% of the baseline FLOPs, exit 2 71.6% with 59%, and exit 3 already matches the baseline (75.3% with 84% of its FLOPs). EE-ViT-Tiny, trained from scratch, reaches 66.5% at its final exit and 62.5%, 66.3% and 66.6% at exits 1 to 3. It is clearly weaker than the CNN, as expected for a vision transformer without pre-training on a small 32x32 dataset and a short schedule [13], [14]; closing this gap is part of the remaining work.

5.2 Accuracy versus Computation

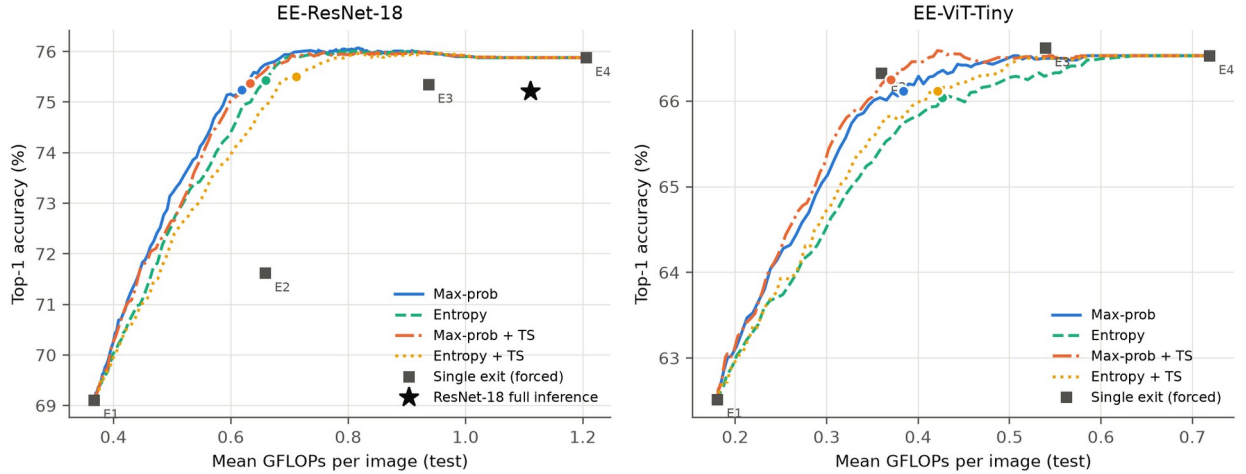


Figure 4: Test accuracy against mean GFLOPs per image while sweeping the exit threshold. Dots mark the thresholds selected on validation; squares are single exits; the star is ResNet-18 with full inference (left).

Table 7: EE-ResNet-18: operating points selected on validation (within 1 point of the final exit).

Exit rule	Threshold	Test acc. (%)	Mean GFLOPs	FLOPs saved vs ResNet-18 (%)	Exits 1/2/3/4 (%)
Max-prob	0.740	75.24	0.619	44.3	54 / 20 / 11 / 16
Entropy	0.840	75.43	0.659	40.7	48 / 20 / 12 / 20
Max-prob + TS	0.660	75.37	0.633	43.1	55 / 16 / 9 / 20
Entropy + TS	0.810	75.50	0.712	35.9	47 / 14 / 10 / 30

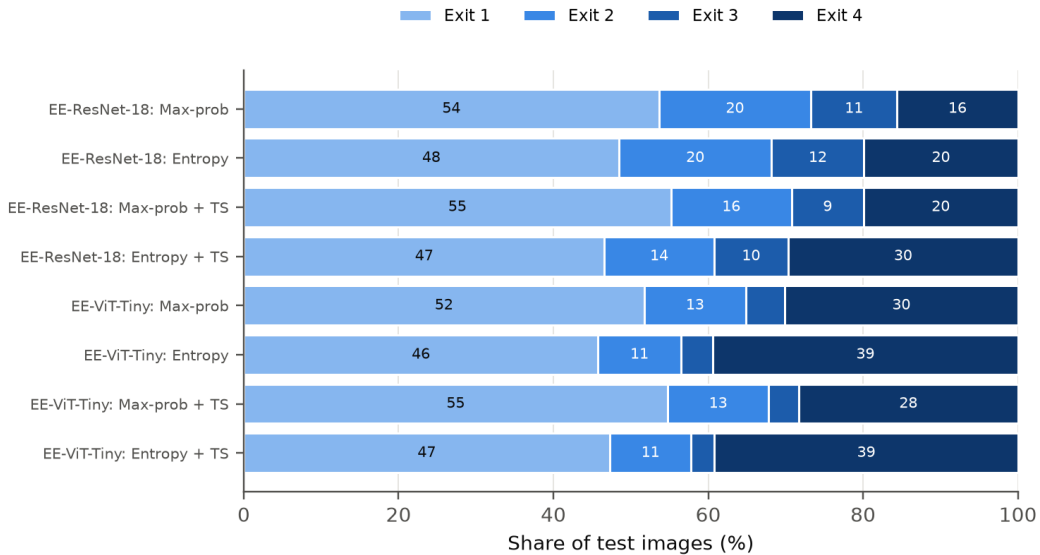
Table 8: EE-ViT-Tiny: operating points selected on validation (within 1 point of the final exit).

Exit rule	Threshold	Test acc. (%)	Mean GFLOPs	FLOPs saved vs ResNet-18 (%)	Exits 1/2/3/4 (%)
Max-prob	0.410	66.12	0.384	65.4	52 / 13 / 5 / 30
Entropy	0.520	66.04	0.427	61.6	46 / 11 / 4 / 39
Max-prob + TS	0.590	66.25	0.370	66.7	55 / 13 / 4 / 28
Entropy + TS	0.740	66.12	0.421	62.1	47 / 11 / 3 / 39

Figure 4 and the tables above contain the central result. With the threshold selected on validation, confidence-based exiting (Model 2) matches the accuracy of full ResNet-18 (75.24% versus 75.22%) while spending on average 0.619 GFLOPs instead of 1.111, a 44.3% reduction; 54% of test images leave at the first exit and only 16% need the full network. The curves saturate early: beyond about 0.7 GFLOPs additional computation brings no accuracy gain. An oracle that stops every image at its earliest correct exit would reach 84.3% accuracy with only 0.56 GFLOPs. Many images are therefore classified correctly by an early exit but wrongly by the final one (overthinking [4]), and a better exit rule has considerable headroom.

The calibrated rules (Model 3) are slightly less efficient on clean data: for similar accuracy, max-prob with temperature scaling needs 0.633 GFLOPs, entropy 0.659 and entropy with temperature scaling 0.712 GFLOPs, and at a budget of 0.55 GFLOPs raw max-prob is ahead by up to 1.2 points. Temperature scaling preserves the ranking of images within an exit and only changes how confidences compare across exits. Since deeper exits receive larger temperatures (Section 5.3), they become less willing to stop images, which pushes images to the costly final exit. Calibration makes confidences faithful, but this is not the same objective as minimising cost at fixed accuracy; tuning per-exit thresholds directly for the budget is a natural next step.

For EE-ViT-Tiny, max-prob exiting stays within one point of its final exit (66.12% versus 66.53%) with 0.384 of its 0.719 GFLOPs (46.5% fewer), and exit 2, after only six of twelve blocks, is already within 0.2 points of the final exit. Here temperature scaling gives the cheapest operating point (0.370 GFLOPs at 66.25%), the opposite of the CNN: the ViT exits are underconfident (temperatures below 1, Section 5.3), so scaling sharpens their confidences and lets more images stop early. Calibration therefore helps routing when exits are underconfident and costs computation when deeper exits are overconfident.

**Figure 5: Share of test images leaving at each exit at the selected thresholds.**

5.3 Calibration of the Exits

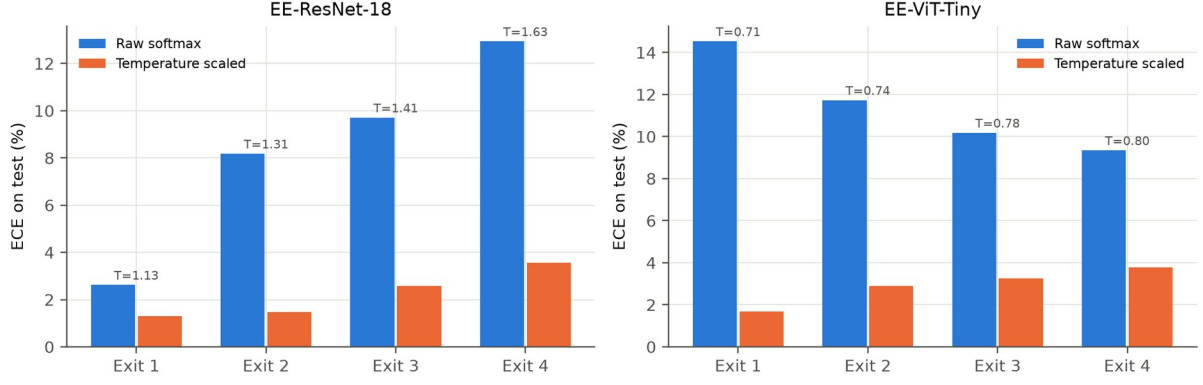


Figure 6: Expected calibration error of each exit before and after temperature scaling (temperatures fitted on validation).

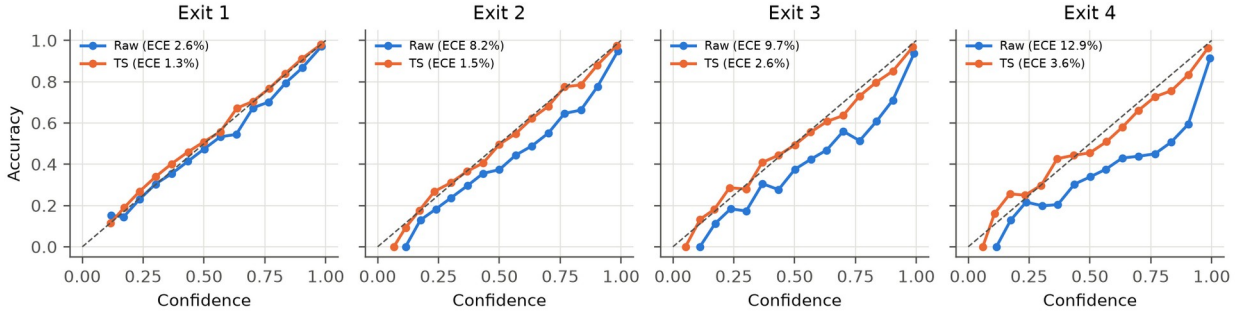


Figure 7: Reliability diagrams of the EE-ResNet-18 exits on the test set.

Raw calibration error grows with depth: exit 1 is nearly calibrated (ECE 2.6%), while the final exit has an ECE of 12.9%, the typical overconfidence of a deep network trained to near-zero training loss [5]. The fitted temperatures increase accordingly from 1.13 to 1.63. Temperature scaling reduces the ECE of every exit, to between 1.3% and 3.6%, and the reliability curves of the deep exits move from below the diagonal (overconfident) close to it. EE-ViT-Tiny behaves differently: trained with label smoothing and mixup, its raw ECE is 9.3% to 14.5% and its temperatures range from 0.71 to 0.80 (below 1 means the exit was underconfident); after scaling its ECE is 1.7% to 3.8%.

5.4 Accuracy at Fixed Compute Budgets

Table 9: EE-ResNet-18: best test accuracy (%) reachable within a mean compute budget.

Exit rule	≤ 0.40 GFLOPs	≤ 0.55 GFLOPs	≤ 0.70 GFLOPs	≤ 0.85 GFLOPs
Max-prob	70.24	74.12	75.94	76.07
Entropy	70.03	73.42	75.85	76.02
Max-prob + TS	70.17	73.88	75.81	75.99
Entropy + TS	69.92	72.96	75.48	75.97

Table 10: EE-ViT-Tiny: best test accuracy (%) reachable within a mean compute budget.

Exit rule	≤ 0.40 GFLOPs	≤ 0.55 GFLOPs	≤ 0.70 GFLOPs	≤ 0.85 GFLOPs
Max-prob	66.20	66.53	66.53	66.53
Entropy	65.82	66.33	66.53	66.53
Max-prob + TS	66.46	66.59	66.59	66.59
Entropy + TS	65.99	66.51	66.53	66.53

At 0.40 GFLOPs (36% of the baseline cost) EE-ResNet-18 still reaches about 70%, and at 0.70 GFLOPs (63%) every rule exceeds full ResNet-18.

5.5 Measured Latency

Table 11: Batch-1 latency (ms per image) when every image is forced to stop at a given exit. GPU: Tesla T4; CPU: Intel(R) Xeon(R) CPU @ 2.00GHz (1 thread).

Model	Device	Exit 1	Exit 2	Exit 3	Final exit
ResNet-18	GPU	-	-	-	2.10
ResNet-18	CPU	-	-	-	18.08
EE-ResNet-18	GPU	1.02	2.01	2.87	3.61
EE-ResNet-18	CPU	6.00	10.60	15.38	21.67
EE-ViT-Tiny	GPU	1.53	2.94	4.26	5.63
EE-ViT-Tiny	CPU	3.59	6.87	10.50	13.45

Table 12: Measured batch-1 latency of real early-exit inference and speed-up over full ResNet-18 (GPU on the first 2,000 and CPU on the first 300 test images).

Model	Exit rule	Threshold	GPU ms	GPU speed-up	CPU ms	CPU speed-up
EE-ResNet-18	Max-prob	0.80	2.23	0.94x	10.75	1.68x
EE-ResNet-18	Max-prob	0.90	2.43	0.86x	12.68	1.43x
EE-ResNet-18	Max-prob + TS	0.80	2.34	0.90x	12.32	1.47x
EE-ResNet-18	Max-prob + TS	0.90	2.58	0.81x	13.80	1.31x
EE-ResNet-18	Entropy + TS	0.80	2.40	0.87x	11.68	1.55x
EE-ResNet-18	Entropy + TS	0.90	2.81	0.75x	14.67	1.23x
EE-ViT-Tiny	Max-prob	0.80	4.71	0.44x	10.72	1.69x
EE-ViT-Tiny	Max-prob	0.90	5.12	0.41x	11.81	1.53x
EE-ViT-Tiny	Max-prob + TS	0.80	3.98	0.53x	8.45	2.14x
EE-ViT-Tiny	Max-prob + TS	0.90	4.40	0.48x	9.62	1.88x
EE-ViT-Tiny	Entropy + TS	0.80	4.08	0.51x	8.41	2.15x
EE-ViT-Tiny	Entropy + TS	0.90	4.78	0.44x	9.69	1.87x

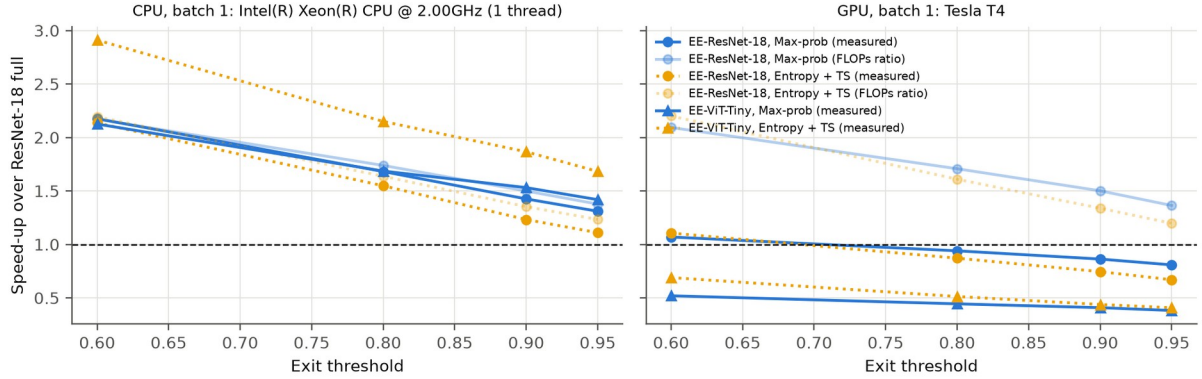


Figure 8: Speed-up over full ResNet-18 at batch 1 as a function of the threshold: measured (solid markers) and predicted from FLOPs (faded).

Measured latency is more nuanced than FLOPs. On one CPU thread, latency follows FLOPs closely: stopping at exit 1 takes 6.0 ms against 18.1 ms for full ResNet-18, and adaptive inference with max-prob at $\tau = 0.8$ needs 10.8 ms per image, a 1.68x speed-up, close to the 1.74x predicted by FLOPs. The full path of the early-exit model is 20% slower than the baseline, which is the price paid by hard images. On the T4 GPU at batch 1 the picture changes: execution is dominated by kernel launches and by the GPU-to-CPU synchronisation needed after every exit to decide whether to stop. Full ResNet-18 takes 2.10 ms and exit 1 alone 1.02 ms, but the full early-exit path takes 3.61 ms, so adaptive inference is at best on par with the baseline (1.07x at $\tau = 0.6$) and slower at higher thresholds (0.86x at $\tau = 0.9$). With batches of 256, where exited images are removed from the batch, the GPU benefits again: throughput rises from 4608 images/s for ResNet-18 to 5750 images/s at $\tau = 0.8$ (1.25x). Early exits therefore pay off on CPUs and edge devices and in batched serving, while FLOP counts overstate the benefit for single images on a GPU, in line with the timing observation of BlockDrop [6]. EE-ViT-Tiny shows the same pattern more strongly. Its full path is cheaper than ResNet-18 on the CPU (13.5 ms) and adaptive inference at $\tau = 0.8$ brings it to 10.7 ms, but on the GPU at batch 1 its twelve blocks launch many small kernels and the full path takes 5.63 ms, 2.7x slower than ResNet-18 despite needing fewer FLOPs.

5.6 Robustness under Corruptions (Preliminary)

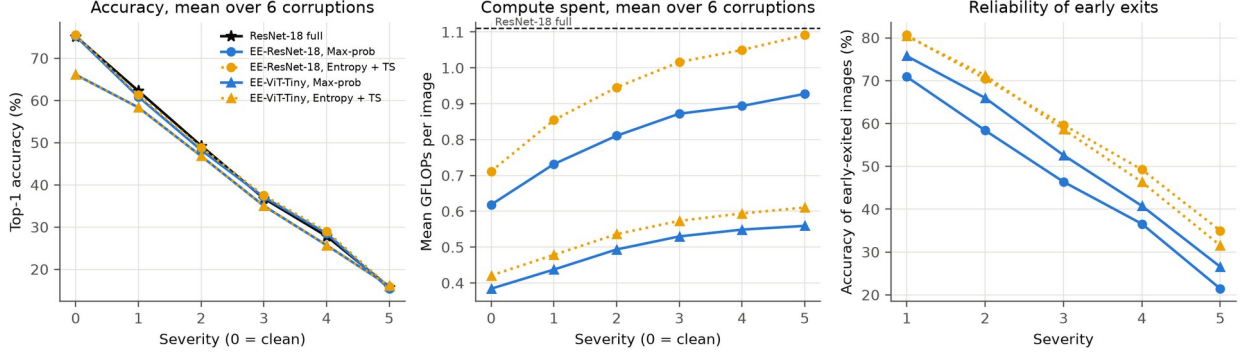


Figure 9: Mean over six corruptions against severity, with thresholds chosen on clean validation data: accuracy (left), compute (middle) and accuracy of the images that exit early (right).

Table 13: Severity 3: accuracy (%) of full ResNet-18 and of EE-ResNet-18 with two exit rules, and mean GFLOPs spent by the early-exit model.

Corruption	ResNet-18 acc.	Max-prob acc.	Max-prob GFLOPs	Entropy+TS acc.	Entropy+TS GFLOPs
Gaussian noise	14.9	12.8	1.02	12.9	1.16
Shot noise	21.9	18.6	0.96	19.2	1.11
Impulse noise	18.7	16.0	0.94	16.4	1.10
Gaussian blur	46.3	50.7	0.80	51.0	0.94
Defocus blur	61.0	63.2	0.72	63.6	0.85
Contrast	58.0	61.3	0.79	61.8	0.93

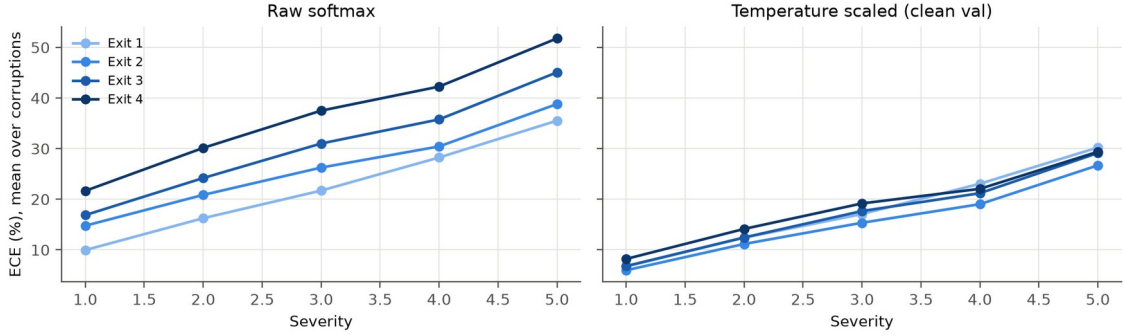


Figure 10: ECE of the EE-ResNet-18 exits under corruption, with raw logits and with temperatures fitted on clean validation data.

With thresholds fixed on clean validation data, the early-exit model keeps the accuracy of full ResNet-18 under corruption: averaged over the six corruptions, max-prob exiting reaches 60.9%, 37.1% and 15.4% at severities 1, 3 and 5, within about one point of the baseline. The model also spends more computation on corrupted images, since fewer of them clear the threshold early: mean cost rises from 0.62 GFLOPs on clean data to 0.93 GFLOPs at severity 5. However, the raw max-prob rule stays overconfident under shift. At severity 5 it still exits 51% of images early, and only 21% of those early decisions are correct. The worst case is impulse noise at severity 5, where accuracy collapses to 3.5% while 69% of images exit early and the model spends less computation than on clean data (0.73 GFLOPs). The calibrated entropy rule is more cautious: it exits 19% of images early at severity 5, of which 35% are correct, at the cost of more computation (1.09 GFLOPs). Temperatures fitted on clean data transfer only partly: averaged over corruptions, the ECE of the final exit is 51.8% with raw logits and still 29.4% after scaling at severity 5 (Figure 10), compared with 3.6% on clean data.

5.7 Summary of Preliminary Findings

- **Compute.** Confidence-based EE-ResNet-18 matches full ResNet-18 accuracy (75.24% vs 75.22%) with 44% fewer FLOPs; its final exit is +0.66 points above the baseline, and exit 1 alone gives 69.1% at 33% of the cost.

- **Headroom.** An oracle exit would reach 84.3% at 0.56 GFLOPs, so better exit rules (and reducing overthinking) can still help.
- **Calibration.** Deeper exits are increasingly overconfident (ECE 2.6% at exit 1 to 12.9% at the final exit); temperature scaling lowers ECE to at most 3.6%.
- **Exit rule.** On clean data, calibration does not improve the accuracy-compute trade-off of a single global threshold; under corruption, the calibrated entropy rule makes early decisions far more reliable, at the price of extra computation.
- **Latency.** FLOP savings become real speed-ups on a CPU (1.68x at $\tau = 0.8$) and in batched GPU inference (1.25x), but not for single images on a GPU, where per-exit synchronisation dominates.
- **Transformer.** Early exiting also works for ViT-Tiny (47% fewer FLOPs within one point of its final exit), but its from-scratch accuracy (66.5%) needs improvement before a fair comparison with the CNN.

6. Risks and Plan for the Remaining Work

6.1 Risks and Mitigation

Table 14: Risks for the remaining work.

Risk	Impact	Mitigation
Early-exit heads reduce the accuracy of the final exit (shared backbone must serve shallow and deep heads) [2].	Medium	Compare EE final exit against Model 1; try exit-loss weighting, gradient scaling for early heads, and self-distillation from the final exit.
Shallow exits are weak, so few images exit early and savings are limited.	High	Stronger heads at early exits, self-distillation, longer training; report accuracy at fixed budgets.
ViT-Tiny trained from scratch on 32x32 CIFAR-100 underperforms the CNN, making the comparison unfair.	Medium	Longer schedule and stronger augmentation; optionally fine-tune an ImageNet-pretrained ViT-Tiny and report its cost separately.
FLOP savings do not translate into GPU latency at batch 1 (kernel launch and per-exit synchronisation overhead).	High	Report GPU and CPU separately; reduce synchronisation; study batched inference with exited samples removed; discuss where early exits pay off.
Temperatures fitted on clean data do not transfer to corrupted inputs.	Medium	Measure ECE under corruption; compare per-exit thresholds and corruption-aware calibration.
Single runs give noisy comparisons between exit rules.	Medium	Repeat final experiments with three seeds and report mean and standard deviation.
Kaggle GPU quota (about 30 h per week) and 12 h session limit.	Low	Two models per session on 2 x T4; checkpoints every 10 epochs; logits stored so analysis is offline.
Latency noise on shared cloud machines.	Low	Warm-up, many images, fixed thread count, repeated runs.

6.2 Remaining Work

- **Model 1 (Raunak Panshikar).** Train the baseline with a longer schedule (200 epochs) and three seeds; extend the latency study to several CPU thread counts and batch sizes.
- **Model 2 (Aditya Goyal).** Improve early-exit accuracy with exit-loss weighting and self-distillation; evaluate per-exit thresholds and the overthinking rate (images correct early but wrong at the end).
- **Model 3 (Arth Mishra).** Compare temperature scaling with vector scaling and a patience-based rule; fit calibration under corruption and study whether calibration chosen on clean data remains valid.
- **Model 4 (Kartikey Goyal).** Retrain ViT-Tiny with a longer schedule and stronger augmentation; try distillation from the final exit to early exits as in LGViT [9]; measure latency.
- **Joint.** Complete the corruption study (all severities, optionally the official CIFAR-100-C files for a cross-check), final comparison of the four models, report and presentation.

6.3 Timeline to Final Submission

Table 15: Updated work plan.

Sl. No.	Work element / milestone	Expected completion
1	Literature review, dataset preparation and preprocessing	Done
2	Baseline ResNet-18 and confidence-based early-exit implementation	Done
3	Calibrated early-exit and ViT-Tiny implementation	Done
4	Preliminary evaluation and interim report	Done (04/10/2026)
5	Tuned training of all models (longer schedules, distillation, 3 seeds)	11/10/2026
6	Corruption experiments, threshold tuning and latency analysis	17/10/2026
7	Ablations (head design, number of exits) and final comparison	23/10/2026

Sl. No.	Work element / milestone	Expected completion
8	Final report and presentation preparation	29/10/2026
9	Final submission	31/10/2026

References

- [1] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in Proc. 23rd Int. Conf. Pattern Recognit. (ICPR), 2016, pp. 2464-2469, doi: 10.1109/ICPR.2016.7900006.
- [2] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Q. Weinberger, "Multi-scale dense networks for resource efficient image classification," in Proc. Int. Conf. Learn. Represent. (ICLR), 2018, arXiv:1703.09844.
- [3] X. Wang, F. Yu, Z.-Y. Dou, T. Darrell, and J. E. Gonzalez, "SkipNet: Learning dynamic routing in convolutional networks," in Proc. Eur. Conf. Comput. Vis. (ECCV), Springer, 2018, pp. 420-436, doi: 10.1007/978-3-030-01261-8_25.
- [4] Y. Kaya, S. Hong, and T. Dumitras, "Shallow-deep networks: Understanding and mitigating network overthinking," in Proc. 36th Int. Conf. Mach. Learn. (ICML), PMLR vol. 97, 2019, pp. 3301-3310.
- [5] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. 34th Int. Conf. Mach. Learn. (ICML), PMLR vol. 70, 2017, pp. 1321-1330.
- [6] Z. Wu, T. Nagarajan, A. Kumar, S. Rennie, L. S. Davis, K. Grauman, and R. Feris, "BlockDrop: Dynamic inference paths in residual networks," in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 8817-8826, doi: 10.1109/CVPR.2018.00919.
- [7] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in Proc. Int. Conf. Learn. Represent. (ICLR), 2019, arXiv:1903.12261.
- [8] M. Wolczyk, B. Wojcik, K. Balazy, I. T. Podolak, J. Tabor, M. Smieja, and T. Trzcinski, "Zero time waste: Recycling predictions in early exit neural networks," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 34, 2021, pp. 2516-2528.
- [9] G. Xu, J. Hao, L. Shen, H. Hu, Y. Luo, H. Lin, and J. Shen, "LGViT: Dynamic early exiting for accelerating vision transformer," in Proc. 31st ACM Int. Conf. Multimedia (MM), 2023, pp. 9103-9114, doi: 10.1145/3581783.3611762.
- [10] L. Meronen, M. Trapp, A. Pilzer, L. Yang, and A. Solin, "Fixing overconfidence in dynamic neural networks," in Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV), 2024, pp. 2668-2678, doi: 10.1109/WACV57701.2024.00266.
- [11] A. Krizhevsky, "Learning multiple layers of features from tiny images," Tech. Rep., Univ. of Toronto, 2009.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2016, pp. 770-778.
- [13] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. Int. Conf. Learn. Represent. (ICLR), 2021.
- [14] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jegou, "Training data-efficient image transformers and distillation through attention," in Proc. 38th Int. Conf. Mach. Learn. (ICML), PMLR vol. 139, 2021, pp. 10347-10357.
- [15] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, "mixup: Beyond empirical risk minimization," in Proc. Int. Conf. Learn. Represent. (ICLR), 2018.
- [16] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, "CutMix: Regularization strategy to train strong classifiers with localizable features," in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), 2019, pp. 6023-6032.